

Static timing analysis with OpenSTA

What it does Calculates whether signals arrive early enough for setup checks and late enough for hold checks. It identifies the paths that need attention.

Remember this OpenSTA measures timing. It does not itself resize gates or build a clock tree. Its results guide those optimization engines.

The three times to understand

Arrival time When data reaches a pin, according to the delay model.

Required time The latest allowed arrival for setup, or the earliest allowed arrival for hold. Clock timing and constraints determine these limits.

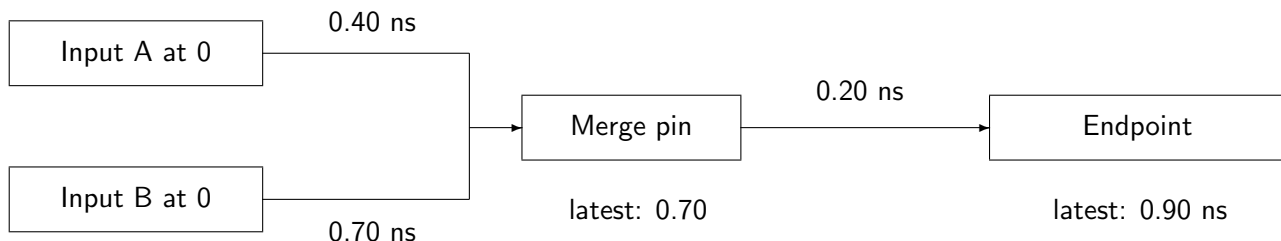
Slack The margin between arrival and the limit. Negative slack means that the timing check fails.

The algorithm is graph propagation

The timing graph has pins as nodes and cell or wire delays as edges. The tool propagates values through this graph instead of simulating every possible input waveform.

- 1 Read the timing model.** Use the netlist, Liberty cell delays, SDC timing constraints and estimated or extracted wire resistance and capacitance.
- 2 Propagate arrival times forward.** Add delay along each edge. At a merge, keep the latest relevant arrival for setup and the earliest relevant arrival for hold.
- 3 Propagate required times backward.** Start from endpoint limits and work backward through delays. This finds how much time is available at upstream pins.
- 4 Calculate slack and report.** Report critical paths, WNS and TNS. After a circuit change, incremental analysis updates affected timing values.

A tiny timing graph



For this simplified setup example, the merge takes $\max(0.40, 0.70)$. The final arrival is $0.70 + 0.20 = 0.90$ ns. Real analysis also distinguishes clocks, transitions and analysis conditions.

Static timing analysis with OpenSTA

Setup and hold use different slack signs

Setup: data must not be too late.

$$\text{setup slack} = \text{required time} - \text{latest arrival time}$$

If the latest allowed arrival is 0.80 ns and data arrives at 0.90 ns, slack is $0.80 - 0.90 = -0.10$ ns. The data is 0.10 ns late.

Hold: new data must not arrive too early.

$$\text{hold slack} = \text{earliest arrival time} - \text{required time}$$

If new data must not arrive before 0.18 ns but arrives at 0.12 ns, slack is $0.12 - 0.18 = -0.06$ ns. We need more minimum data-path delay.

WNS tells us the worst violation

WNS means worst negative slack. Consider four endpoint setup slacks:

$$A : -0.20 \quad B : -0.10 \quad C : +0.05 \quad D : -0.05 \quad \text{ns}$$

The WNS is -0.20 ns. Endpoint A has the largest timing failure.

TNS tells us the accumulated violation

TNS means total negative slack. Sum only the negative endpoint slacks:

$$\text{TNS} = -0.20 - 0.10 - 0.05 = -0.35 \text{ ns}$$

For the selected analysis, use each endpoint's worst relevant slack. Do not add every enumerated path to the same endpoint, and do not let positive slack cancel negative slack. Setup and hold are reported separately.

What zero means

OpenSTA's `report_wns` clamps positive worst slack to zero. If all checked endpoints pass, WNS and TNS are both zero. `report_worst_slack` can still show a positive margin. Correct constraints and all required operating corners must also be checked; zero in one report alone is not complete timing closure.

Commands and a short spoken explanation

```
report_checks -path_delay max
```

```
report_wns -max      report_tns -max
```

Use `-min` for hold WNS/TNS; a detailed hold report uses `-path_delay min`.

"OpenSTA propagates arrival and required times through a timing graph. Slack tells us whether a check passes. WNS is the worst negative margin; TNS adds the negative margins across endpoints."

Sources and code Official OpenSTA repository and documentation. Definitions checked against `src/sta/search/Search.tcl` and `Search.cc` in the local OpenROAD checkout. Values above are teaching examples with already determined required times.